Αναφορά 5.1

Repository: https://github.com/gkalogeitonas/Agronos-iot-device

Φοιτητής: Καλογείτονας Γεωργιος

Επιβλέπων Καθηγητής: Νικόλαος Σκλάβος

Ημερομηνία: Μάρτιος 2026

Περίληψη

Η παρούσα αναφορά καταγράφει την υλοποίηση του **firmware κόμβου LoRa** για την πλατφόρμα Agronos. Αναπτύχθηκε ένας νέος, ανεξάρτητος τύπος συσκευής που λειτουργεί αποκλειστικά μέσω LoRa radio, χωρίς χρήση WiFi. Ο κόμβος διαβάζει αισθητήρες, σειριοποιεί τα δεδομένα, τα κρυπτογραφεί με AES-128-CTR και τα μεταδίδει στο LoRa Gateway, το οποίο τα προωθεί στο backend. Όλα τα κρυπτογραφικά πρωτόκολλα έχουν σχεδιαστεί ώστε να είναι πλήρως συμβατά με την `LoRaCryptoService` που υλοποιήθηκε στο Laravel (Αναφορά 5).

Αρχιτεκτονική Επικοινωνίας LoRa

Η πλήρης ροή δεδομένων από τον κόμβο έως το backend είναι:

```
[LoRa Node - TTGO LoRa32]
        │   LoRa Radio (868MHz, SF10)
        │   Πακέτο: [1B uuid_len][UUID bytes][4B fcnt][AES-CTR ciphertext]
        ▼
[LoRa Gateway - ESP32]
        │   MQTT   topic: lora/{gateway_uuid}/data
        │   JSON:  { gateway_id, rssi, snr, raw_payload (hex) }
        ▼
[EMQX Broker]
        │   Rule Engine: topic filter lora/+/data
        │   HTTP POST → /api/v1/lora/webhook
        ▼
[Laravel Backend]
        Αποκρυπτογράφηση → Αποσειριοποίηση → SensorDataService
```

Βασικό αρχιτεκτονικό χαρακτηριστικό: **Zero-Trust**. Το Gateway βλέπει μόνο κρυπτογραφημένα bytes — ακόμη και αν παραβιαστεί, τα δεδομένα παραμένουν αδιάβαστα.

Νέο PlatformIO Environment: `ttgo-lora32-v21`

Προστέθηκε τρίτο build target στο `platformio.ini`, αποκλειστικά για τη **TTGO LoRa32 V2.1** πλακέτα:

```
[env:ttgo-lora32-v21]
platform = espressif32
board = ttgo-lora32-v21
build_flags =
    -D LORA_NODE=1
lib_deps =
    ${env.lib_deps}
    sandeepmistry/LoRa@^0.8.0
src_filter =
    +<*>
    -<main.cpp>
    -<wifi_portal.cpp>
    -<auth.cpp>
    -<data_sender.cpp>
    -<mqtt_client.cpp>
```

Βασικές αποφάσεις σχεδίασης:

- **`src_filter` αντί `#ifdef`**: Τα WiFi-specific αρχεία εξαιρούνται εντελώς από τον LoRa build target μέσω φίλτρου αρχείων, αντί χρήσης preprocessor directives. Αυτό κρατά τον κώδικα καθαρό και αποτρέπει τη διαρροή WiFi-εξαρτήσεων στον LoRa κόμβο.
- **Build flag `-D LORA_NODE=1`**: Ενεργοποιεί τα LoRa-specific τμήματα του `config.h` (pin definitions, radio settings, frame counter constants).
- **Αμοιβαία εξαίρεση**: Τα WiFi environments (`esp32dev`, `esp32-c6-devkitc-1`) εξαιρούν αντίστροφα τα LoRa αρχεία (`main_lora.cpp`, `lora_*.cpp`) μέσω των δικών τους `src_filter`, ώστε και οι τρεις variants να builds ανεξάρτητα.

Διαμόρφωση LoRa στο `config.h`

Τα LoRa-specific constants ορίζονται υπό `#ifdef LORA_NODE` ώστε να είναι διαθέσιμα μόνο στον κατάλληλο build target:

```
#ifdef LORA_NODE

// LoRa SPI pins (TTGO LoRa32 V2.1)
constexpr int LORA_SCK_PIN   = 5;
constexpr int LORA_MISO_PIN  = 19;
constexpr int LORA_MOSI_PIN  = 27;
constexpr int LORA_SS_PIN    = 18;
constexpr int LORA_RST_PIN   = 14;
constexpr int LORA_DIO0_PIN  = 26;

// LoRa radio settings (must match gateway defaults)
constexpr long    LORA_FREQUENCY  = 868000000; // 868 MHz (Europe)
constexpr int     LORA_SF         = 10;        // Spreading Factor 10
constexpr long    LORA_BW         = 125000;    // 125 kHz bandwidth
constexpr int     LORA_CR         = 5;         // Coding Rate 4/5
constexpr uint8_t LORA_SYNC_WORD  = 0x12;      // Private network sync
```

```
word
constexpr int     LORA_TX_POWER   = 20;          // 20 dBm

// Frame counter management
constexpr uint32_t FCNT_NVS_SAVE_INTERVAL = 100; // Lazy NVS save interval
constexpr uint32_t FCNT_COLD_BOOT_GAP     = 100; // Safety gap on power
loss

#endif // LORA_NODE
```

Σημαντική παρατήρηση: Στη TTGO LoRa32 V2.1, το GPIO 14 χρησιμοποιείται ως `LORA_RST_PIN`. Για αυτό, το button pin για factory reset αλλάζει σε GPIO 0 (BOOT button) όταν `LORA_NODE` είναι ορισμένο:

```
#ifdef LORA_NODE
constexpr int BUTTON_PIN = 0; // BOOT button (GPIO 14 is occupied by LoRa
RST)
#elif defined(CONFIG_IDF_TARGET_ESP32C6)
constexpr int BUTTON_PIN = 7;
#else
constexpr int BUTTON_PIN = 14;
#endif
```

Προφίλ Συσκευής με LoRa Διαπιστευτήρια

Το `device_profile.h` (και το αντίστοιχο `device_profile.h.example`) περιέχει μόνο το AES κλειδί ως LoRa-specific ρύθμιση:

```
// 128-bit AES key (συμπίπτει με το lora_aes_key hex στη βάση δεδομένων)
constexpr uint8_t LORA_AES_KEY[16] = {
    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
    0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F
};
```

Η αναγνώριση της συσκευής γίνεται μέσω του `DEFAULT_UUID` που ήδη ορίζεται στο device profile. Δεν χρειάζεται ξεχωριστό `LORA_DEVICE_ID` — η συσκευή δεν χρειάζεται να γνωρίζει το numeric `devices.id` του backend. Το UUID ενσωματώνεται στο LoRa πακέτο ως header, ώστε το backend να αναγνωρίζει τη συσκευή, και χρησιμοποιείται (μέσω CRC32 hash) για την κατασκευή του nonce.

Δυαδική Σειριοποίηση: `lora_payload.cpp`

Υλοποιήθηκε η συνάρτηση `serializeReadings()` που μετατρέπει τη λίστα των αναγνώσεων αισθητήρων σε compact binary format.

Δομή ανά αισθητήρα (6 bytes):

Bytes	Περιεχόμενο
0–3	Πρώτοι 4 χαρακτήρες (ASCII) του UUID αισθητήρα
4–5	Τιμή × 100 ως `int16_t` Little-Endian

Η δομή αυτή είναι πανομοιότυπη με αυτή που αναμένει η `LoRaCryptoService::deserialize()` στο backend.

Παράδειγμα: Αισθητήρας με UUID `"DHT2-Temp-1"`, τιμή `23.45`:

- Bytes 0-3: `44 48 54 32` (ASCII: "DHT2")
- Bytes 4-5: `0x0929` LE → `29 09` (2345 = 23.45 × 100)

Το ESP32 χρησιμοποιεί εγγενώς Little-Endian αρχιτεκτονική, οπότε ένα απλό `memcpy` είναι αρκετό χωρίς byte swapping:

```
int16_t scaled = static_cast<int16_t>(readings[i].value * 100.0f);
memcpy(chunk + 4, &scaled, sizeof(int16_t));
```

Κρυπτογράφηση AES-128-CTR: `lora_crypto.cpp`

Κατασκευή Nonce

Το 16-byte nonce κατασκευάζεται ντετερμινιστικά από γνωστές τιμές, χωρίς να χρειαστεί μετάδοσή του στον αέρα. Η δομή είναι πανομοιότυπη με αυτή του backend:

```
Nonce[16] = [4B: CRC32(DEFAULT_UUID) LE] [4B: fcnt LE] [8B: 0x00]
```

Η `uuidHash()` υπολογίζει standard CRC32 (polynomial `0xEDB88320`), πανομοιότυπο με τη PHP `crc32()`, ώστε firmware και backend να παράγουν ακριβώς το ίδιο nonce.

```
uint32_t uuidHash(const char* uuid) {
    uint32_t crc = 0xFFFFFFFF;
    while (*uuid) {
        crc ^= (uint8_t)*uuid++;
        for (int j = 0; j < 8; ++j) {
            crc = (crc >> 1) ^ (0xEDB88320 & -(crc & 1));
        }
    }
    return crc ^ 0xFFFFFFFF;
}

void buildNonce(const char* uuid, uint32_t fcnt, uint8_t nonce[16]) {
    memset(nonce, 0, 16);
    uint32_t hash = uuidHash(uuid);
    memcpy(nonce, &hash, 4);   // Little-Endian (native on ESP32)
```

```
    memcpy(nonce + 4, &fcnt, 4);
}
```

Κρυπτογράφηση

Χρησιμοποιείται η βιβλιοθήκη `mbedtls`. Η λειτουργία AES-CTR παράγει ciphertext ακριβώς ίδιου μεγέθους με το plaintext

```
mbedtls_aes_crypt_ctr(&ctx, len, &ncOffset, nonceCounter, streamBlock,
                      plaintext, ciphertext);
```

Η χρήση του hardware-accelerated `mbedtls` εξασφαλίζει:

1. **Ταχύτητα**: Ο ESP32 διαθέτει hardware AES accelerator
2. **Ασφάλεια**: Καθιερωμένη βιβλιοθήκη, χωρίς custom κρυπτογραφικό κώδικα
3. **Μηδενικές επιπλέον εξαρτήσεις**: Ήδη διαθέσιμη στο PlatformIO/ESP-IDF

Διαχείριση Frame Counter: `lora_fcnt.cpp`

Ο frame counter (`fcnt`) είναι μονοτονικά αυξανόμενος αριθμός που χρησιμοποιείται για αντι-replay προστασία. Η υλοποίηση επιλύει δύο αντικρουόμενες ανάγκες: **ακρίβεια** (να μην επαναχρησιμοποιηθεί ποτέ ο ίδιος counter) και **προστασία flash** (να μην γράφεται στην NVS σε κάθε μετάδοση).

Υβριδική Μνήμη RTC + NVS

```
RTC_DATA_ATTR static uint32_t rtcFcnt = 0;         // Επιβιώνει το Deep
Sleep
RTC_DATA_ATTR static bool     rtcInitialized = false;
RTC_DATA_ATTR static uint32_t txSinceLastSave = 0;  // Lazy save counter
```

Σενάριο	Συμπεριφορά
Deep Sleep Wake	Η `rtcInitialized` είναι `true` → η RTC τιμή είναι έγκυρη, καμία NVS I/O
Cold Boot (power loss)	Διαβάζει NVS, προσθέτει `FCNT_COLD_BOOT_GAP=100`, αποθηκεύει στο NVS
Κάθε 100 TX	Lazy save: αποθηκεύει την τρέχουσα RTC τιμή στο NVS

Το άλμα +100 σε cold boot εξασφαλίζει ότι ακόμη και αν η τελευταία NVS εγγραφή ήταν 99 μεταδόσεις πριν από την απώλεια ρεύματος, ο κόμβος επανεκκινεί με counter **πάνω** από τον τελευταίο μεταδοθέντα. Αυτό είναι ασφαλές γιατί το backend επιτρέπει άλμα έως `MAX_FCNT_GAP = 10.000`.

LoRa Radio Driver: `lora_radio.cpp`

Ο driver διαχειρίζεται την αρχικοποίηση και μετάδοση μέσω της βιβλιοθήκης `sandeepmistry/LoRa`.

Αρχικοποίηση

```
bool loraRadioInit() {
    SPI.begin(LORA_SCK_PIN, LORA_MISO_PIN, LORA_MOSI_PIN, LORA_SS_PIN);
    LoRa.setPins(LORA_SS_PIN, LORA_RST_PIN, LORA_DIO0_PIN);
    LoRa.begin(LORA_FREQUENCY);
    LoRa.setSpreadingFactor(LORA_SF);      // SF10
    LoRa.setSignalBandwidth(LORA_BW);      // 125 kHz
    LoRa.setCodingRate4(LORA_CR);          // 4/5
    LoRa.setSyncWord(LORA_SYNC_WORD);      // 0x12 (private network)
    LoRa.setTxPower(LORA_TX_POWER);        // 20 dBm
}
```

Δομή Πακέτου Μετάδοσης

```
bool loraTransmit(const char* uuid, uint32_t fcnt,
                  const uint8_t* ciphertext, size_t cipherLen) {
    uint8_t uuidLen = (uint8_t)strlen(uuid);
    uint8_t fcntBytes[4];
    memcpy(fcntBytes, &fcnt, 4);        // ESP32 is natively LE

    LoRa.beginPacket();
    LoRa.write(uuidLen);                // Byte 0: UUID length
    LoRa.write((const uint8_t*)uuid, uuidLen);  // Bytes 1..N: UUID (ASCII)
    LoRa.write(fcntBytes, 4);           // Bytes N+1..N+4: Frame Counter
(LE)
    LoRa.write(ciphertext, cipherLen); // Bytes N+5..: Ciphertext
    LoRa.endPacket(true);               // Blocking TX
}
```

Η νέα δομή πακέτου `[1B uuid_len][UUID bytes][4B fcnt LE][ciphertext]` επιτρέπει στο backend να αναγνωρίζει τη συσκευή απευθείας από το UUID.

Το Gateway λαμβάνει αυτό το πακέτο ως `raw[]` μέσω `loraPoll()`, το κωδικοποιεί σε hex string και το εμπεριέχει στο JSON προς τον EMQX Broker. Το backend αποκωδικοποιεί το hex, εξάγει πρώτα το UUID length, μετά το UUID, τον frame counter, και αποκρυπτογραφεί το υπόλοιπο.

Κύρια Ροή Εκτέλεσης: `main_lora.cpp`

Ο κόμβος εκτελεί **μία μόνο iteration** ανά αφύπνιση. Η συνάρτηση `loop()` παραμένει κενή.

```
setup()
  │
  ├─ checkButtonReset()      → Factory reset αν κουμπί κρατηθεί >10 δευτ.
  ├─ fcntInit(storage)       → RTC warm wake ή NVS cold boot recovery
  ├─ loraRadioInit()         → SPI init + LoRa settings
```

```
    ├─ createSensors()          → Factory pattern (κοινός με WiFi variant)
    ├─ sensors[i]->read()       → Ανάγνωση τιμών
    ├─ serializeReadings()      → Binary payload (N × 6 bytes)
    ├─ fcntNext(storage)        → Αύξηση counter + lazy NVS save
    ├─ buildNonce()             → [CRC32(UUID) LE][fcnt LE][zeros]
    ├─ encryptPayload()         → AES-128-CTR via mbedtls
    ├─ loraTransmit()           → [1B uuid_len][UUID][4B fcnt][ciphertext]
    ├─ loraRadioSleep()         → LoRa radio σε sleep mode
    └─ esp_deep_sleep_start()   → ESP32 deep sleep για
  SENSORS_READ_INTERVAL_MS
```

Σε περίπτωση αποτυχίας σε οποιοδήποτε βήμα (init radio, serialization, encryption), ο κόμβος εισέρχεται σε deep sleep χωρίς μετάδοση, αποφεύγοντας τη σπατάλη ρεύματος.

Επαναχρησιμοποίηση Υπαρχόντων Υποδομών

Βασικό αρχιτεκτονικό πλεονέκτημα: οι LoRa κόμβοι **επαναχρησιμοποιούν** το σύνολο της υπάρχουσας υποδομής αισθητήρων:

Υποδομή	WiFi Variant	LoRa Variant
`sensor.h` / `SensorBase`	✓	✓
`sensor_factory.cpp`	✓	✓
`dht11_temp.cpp`, `dht11_hum.cpp`	✓	✓
`dht20_sensors.cpp`	✓	✓
`soil_moisture.cpp`	✓	✓
`battery_level.cpp`	✓	✓
`storage.cpp` (NVS)	✓	✓
`wifi_portal.cpp`	✓	✗
`auth.cpp` / `data_sender.cpp`	✓	✗
`lora_*.cpp`	✗	✓

Αυτό σημαίνει ότι η προσθήκη νέων αισθητήρων ακολουθεί ακριβώς την ίδια **3-step διαδικασία** και για τους δύο τύπους συσκευών.

Σχετικά Αρχεία

Νέα αρχεία:

- src/main_lora.cpp — LoRa-only entry point
- src/lora_payload.cpp / include/lora_payload.h — Binary serializer
- src/lora_crypto.cpp / include/lora_crypto.h — AES-128-CTR + Nonce
- src/lora_fcnt.cpp / include/lora_fcnt.h — Frame Counter Manager

- src/lora_radio.cpp / include/lora_radio.h — LoRa Radio Driver

Τροποποιημένα αρχεία:

- platformio.ini — Προσθήκη `[env:ttgo-lora32-v21]`, src_filter σε όλα τα environments
- include/config.h — LoRa pin definitions, radio settings, fcnt constants, button pin fix
- include/device_profile.h.example — `LORA_AES_KEY`
- include/storage.h / src/storage.cpp — `getLoraFcnt()`, `setLoraFcnt()`, επέκταση `clearAll()`

LoRa Gateway

*Repository:** https://github.com/gkalogeitonas/ESP32-LoRa-to-Internet-Gateway

Η ανάγκη για τη δημιουργία του LoRa Gateway προέκυψε από την πρακτική απαίτηση να γεφυρωθεί το τοπικό LoRa δίκτυο με το Internet χωρίς το υψηλό κόστος και την πολυπλοκότητα ενός πλήρους LoRaWAN concentrator. Το gateway λειτουργεί ως ελαφρύ transport-layer bridge: λαμβάνει αδιαφανή LoRa πακέτα, τα εμπλουτίζει με ραδιομεταδεδομένα και τα προωθεί προς MQTT ή HTTP, χωρίς να αποκρυπτογραφεί το περιεχόμενο. Αυτή η προσέγγιση μειώνει δραστικά το κόστος υλοποίησης και συντήρησης (hardware, provisioning, και λειτουργική πολυπλοκότητα), επιτρέπει γρήγορη επιτόπια εγκατάσταση και παραμετροποίηση και είναι ιδιαίτερα κατάλληλη για μικρής έως μεσαίας κλίμακας αγροτικές εφαρμογές. Παράλληλα, διατηρείται το μοντέλο ασφαλείας “zero-trust”: τα κρυπτογραφημένα δεδομένα παραμένουν αδιάβλητα μέχρι το backend, όπου γίνεται η αποκρυπτογράφηση και η επεξεργασία. Με αυτόν τον τρόπο το gateway προσφέρει έναν πρακτικό, οικονομικό και ασφαλή ενδιάμεσο κρίκο ανάμεσα στους LoRa κόμβους και την πλατφόρμα Agronos.

Επιπλέον, το LoRa Gateway επιλέχθηκε και αναπτύχθηκε ως ανεξάρτητο project, ξεχωριστό από το κύριο repository του Agronos, ώστε να λειτουργεί ως γενικό, επαναχρησιμοποιήσιμο εργαλείο που μπορεί να αξιοποιηθεί από την πλατφόρμα Agronos και από άλλα συστήματα· αυτή η επιλογή επιτρέπει ανεξάρτητη συντήρηση, μεγαλύτερη ευελιξία στην ανάπτυξη και σημαντικά μειωμένο κόστος εγκατάστασης σε σχέση με την υλοποίηση ενός πλήρους LoRaWAN concentrator.

Hardware Target και PlatformIO Environment

Το project στοχεύει αποκλειστικά την **TTGO LoRa32 v2.1**, η οποία ενσωματώνει:

- **ESP32-D0WDQ6** MCU
- **SX1276** LoRa transceiver (868 MHz έκδοση)
- **SSD1306** OLED οθόνη μέσω I2C
- Είσοδο μέτρησης μπαταρίας μέσω GPIO35
- Υποστήριξη LiPo τροφοδοσίας

Το αντίστοιχο PlatformIO environment είναι:

```
[env:ttgo-lora32-v21]
platform = espressif32
board = ttgo-lora32-v21
framework = arduino

monitor_speed = 115200
```

```
board_build.filesystem = littlefs
board_build.partitions = huge_app.csv
```

Οι κύριες βιβλιοθήκες που χρησιμοποιούνται είναι:

- `sandeepmistry/LoRa` για SX1276 radio handling
- `ESP Async WebServer` και `AsyncTCP` για το captive portal
- `ArduinoJson` για config και payload serialization
- `PubSubClient` για MQTT forwarding
- `ESP32 OLED driver for SSD1306` για το dashboard

Runtime Αρχιτεκτονική

Η εκκίνηση του firmware στο `setup()` ακολουθεί συγκεκριμένη σειρά αρχικοποίησης:

```
configInit()
  → displayInit()
  → batteryInit()
  → loraInit(gConfig)
  → wifiInit(gConfig)
  → wifiScanSync()
  → webServerInit()
  → forwarderInit(gConfig)
```

Η σειρά αυτή είναι κρίσιμη διότι:

- το `configInit()` πρέπει να προηγηθεί ώστε να φορτωθεί το `gConfig`
- το `displayInit()` χρησιμοποιείται από νωρίς για boot/status μηνύματα
- το `loraInit()` εξαρτάται από τα LoRa πεδία του config
- το `wifiInit()` αποφασίζει αν η συσκευή θα μπει σε STA ή AP mode
- το `webServerInit()` ξεκινά αφού υπάρχει διαθέσιμο δίκτυο ή captive AP
- το `forwarderInit()` εξαρτάται από το αν έχει επιλεγεί MQTT ή HTTP

Το `loop()` ακολουθεί cooperative αρχιτεκτονική χωρίς RTOS tasks. Οι επιμέρους λειτουργίες εκτελούνται περιοδικά μέσω `millis()`-based cadence:

- battery polling κάθε 30 s
- OLED refresh κάθε 1 s
- MQTT keepalive κάθε 100 ms
- WiFi reconnect attempt κάθε 60 s
- LoRa packet polling σε κάθε iteration

Η επιλογή αυτή κρατά το firmware απλό, ντετερμινιστικό και εύκολο να τεκμηριωθεί ακαδημαϊκά.

Διαχείριση Ρυθμίσεων: `config.cpp`

Η δομή `GatewayConfig` αποτελεί το μοναδικό σημείο αλήθειας για τη runtime κατάσταση του gateway. Περιλαμβάνει:

- WiFi SSID και password
- LoRa frequency, spreading factor, bandwidth, coding rate, sync word
- επιλογή forwarding mode (`use_mqtt`)
- MQTT broker, port, topic, credentials
- HTTP endpoint URL
- `gateway_id`

Οι ρυθμίσεις αποθηκεύονται στο αρχείο `/config.json` στο LittleFS. Αν το αρχείο δεν υπάρχει, εκτελείται `configDefaults()` και δημιουργείται νέο default configuration.

Captive Portal και Web-Based Παραμετροποίηση

Το gateway διαθέτει ενσωματωμένο web interface για επιτόπια ρύθμιση χωρίς re-flash. Αν δεν υπάρχουν έγκυρα WiFi credentials, η συσκευή ενεργοποιεί Access Point με SSID:

```
LoRa-GW-Setup
```

Παράλληλα ενεργοποιείται `DNSServer` wildcard redirection, ώστε οι περισσότερες συσκευές να οδηγούνται αυτόματα στο captive portal.

Ενότητες Web UI

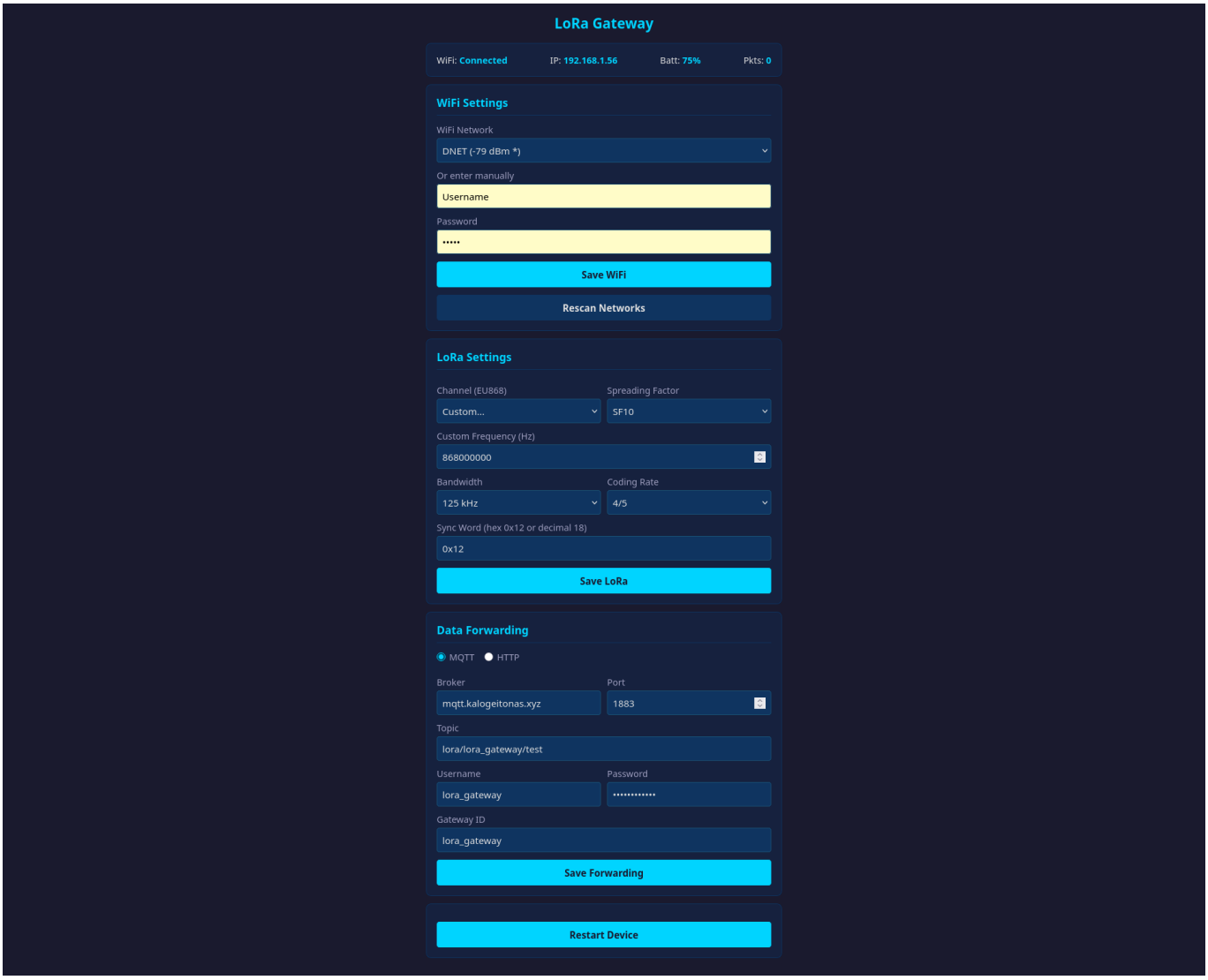
Η σελίδα configuration περιλαμβάνει τρεις βασικές ενότητες:

1. **WiFi Settings**
    - επιλογή SSID από scan
    - manual εισαγωγή SSID
    - password
    - rescan networks
2. **LoRa Settings**
    - έτοιμα EU868 channel presets
    - custom frequency input
    - spreading factor (`SF7`–`SF12`)
    - bandwidth (`125/250/500 kHz`)
    - coding rate (`4/5` έως `4/8`)
    - sync word
3. **Data Forwarding**
    - επιλογή MQTT ή HTTP
    - MQTT broker / port / topic / credentials
    - HTTP POST URL
    - `gateway_id`

Οι HTTP handlers δεν εφαρμόζουν βαριές αλλαγές απευθείας. Αποθηκεύουν το νέο config και θέτουν flags όπως:

- `flagWifiChanged`
- `flagLoraChanged`
- `flagForwardChanged`
- `flagRestartRequested`
- `flagRescanRequested`

Η πραγματική επαναδιαμόρφωση γίνεται αργότερα από το `loop()` του `main.cpp`. Αυτή η απόφαση αποφεύγει blocking work μέσα σε async web callbacks και κρατά σταθερή τη συμπεριφορά του web server.



Εικόνα: Η OLED οθόνη του gateway με ένδειξη WiFi, battery, SF/RSSI, και counters Rx/Fwd.

LoRa Radio Driver: `lora_radio.cpp`

Το υποσύστημα LoRa βασίζεται στη βιβλιοθήκη `LoRa` του Sandeep Mistry και αρχικοποιεί το SPI bus και τα radio pins της TTGO LoRa32.

Η αρχικοποίηση περιλαμβάνει:

```
```

```
SPI.begin(LORA_SCK_PIN, LORA_MISO_PIN, LORA_MOSI_PIN, LORA_SS_PIN);
LoRa.setPins(LORA_SS_PIN, LORA_RST_PIN, LORA_DIO0_PIN);
LoRa.begin(cfg.lora_frequency);
LoRa.setSpreadingFactor(cfg.lora_sf);
LoRa.setSignalBandwidth(cfg.lora_bandwidth);
LoRa.setCodingRate4(cfg.lora_coding_rate);
LoRa.setSyncWord(cfg.lora_sync_word);
LoRa.receive();
```

Ο driver λειτουργεί αποκλειστικά σε receive mode. Κάθε νέο πακέτο συλλέγεται μέσω `loraPoll()` και αποθηκεύεται σε `LoRaPacket` με τα εξής στοιχεία:

- `raw[]` bytes
- `size`
- `rssi`
- `snr`
- `frequency`
- `sf`
- `timestamp`

Το `timestamp` βασίζεται στο `millis()` του ESP32 και χρησιμοποιείται ως local ingress timestamp του gateway. Δεν αντικαθιστά το application timestamp του backend, αλλά δίνει χρήσιμο διαγνωστικό context.

Προώθηση Δεδομένων: MQTT και HTTP

Το `forwarder.cpp` μετατρέπει κάθε εισερχόμενο LoRa frame σε JSON payload. Το schema είναι:

```
{
  "gateway_id": "lora-gw-001",
  "timestamp": 123456,
  "rssi": -87,
  "snr": 7.5,
  "spreading_factor": 10,
  "frequency": 868100000,
  "packet_size": 14,
  "raw_payload": "010000000A000000D44F8C..."
}
```

`raw_payload` ως Hex String

Το radio payload δεν αποσυντίθεται από το gateway. Μετατρέπεται σε hex string και προωθείται αυτούσιο. Αυτό επιτρέπει:

- μεταφορά binary ciphertext μέσω JSON χωρίς αλλοίωση
- διατήρηση πλήρους συμβατότητας με την anti-replay και decrypt λογική του backend
- αυστηρό διαχωρισμό ευθυνών μεταξύ gateway και εφαρμογικού επιπέδου

MQTT Mode

Αν `use_mqtt = true`, το gateway συνδέεται στον broker με `PubSubClient` και κάνει publish στο configured topic. Στο πλαίσιο της διπλωματικής, η προτεινόμενη μορφή topic είναι:

```
lora/{gateway_id}/data
```

Αυτό ευθυγραμμίζεται πλήρως με τη ροή που περιγράφηκε στην Αναφορά 5.

HTTP Mode

Αν `use_mqtt = false`, το ίδιο JSON στέλνεται μέσω `HTTPClient::POST()` στο configured endpoint. Το mode αυτό είναι χρήσιμο για:

- απλούστερα εργαστηριακά setups
- debugging χωρίς broker
- άμεση σύνδεση με backend webhook ή intermediate service

Η ύπαρξη και των δύο modes επιτρέπει στην ίδια firmware βάση να εξυπηρετεί τόσο το production-oriented architecture του PRD όσο και απλούστερα proof-of-concept σενάρια.

Παρακολούθηση Μπαταρίας και Προστασία Συσκευής

Η μέτρηση της μπαταρίας γίνεται μέσω ADC στο **GPIO35**, αξιοποιώντας τον ενσωματωμένο voltage divider της TTGO πλακέτας. Η μετατροπή τάσης σε ποσοστό βασίζεται σε piecewise linear interpolation επάνω σε καμπύλη εκφόρτισης LiPo.

Όταν το επίπεδο πέσει κάτω από 5%, καλείται:

```
loraDisable();
```

Με αυτόν τον τρόπο η συσκευή σταματά τη λειτουργία του radio και προστατεύει το στοιχείο τροφοδοσίας από deep discharge. Η κατάσταση αυτή προβάλλεται και στην OLED ως `LoRa OFF (low batt)`.

OLED Dashboard

Η OLED 128x64 λειτουργεί ως κύριο μέσο επιτόπιας διάγνωσης. Η σελίδα dashboard εμφανίζει:

- εικονίδιο WiFi / AP mode
- IP address ή ένδειξη `AP Mode`
- battery percentage και battery icon
- τελευταία ληφθείσα τιμή `SF / RSSI`
- current channel / configured SF
- counters `Rx` και `Fwd`

Αυτό είναι ιδιαίτερα χρήσιμο στο πεδίο, όπου δεν υπάρχει πάντα πρόσβαση σε serial monitor ή laptop. Ο χρήστης μπορεί να επιβεβαιώσει άμεσα αν:

- το gateway έχει συνδεθεί στο WiFi
- ακούει στο σωστό κανάλι
- λαμβάνει πακέτα
- προωθεί επιτυχώς τα δεδομένα

Εικόνα: Η OLED οθόνη του gateway με ένδειξη WiFi, battery, SF/RSSI, και counters Rx/Fwd.